You said:

Lab_4_VibeCoding...
PDF

https://github.com/SETAPESU26/14_2048 i am supposed to complete this lab work please help me complete this end to end as per the requirements

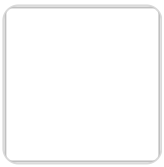
ChatGPT said:

You said:

bro tell me stepwise what the question is what we are doing in each step in short so i know whaats going on and how to do

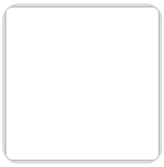
ChatGPT said:

You said:



ChatGPT said:

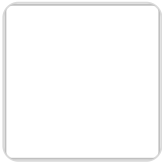
You said:



how do i get back to normal path and not this project path

ChatGPT said:

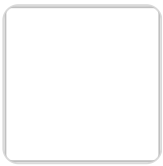
You said:



venv not needed right?

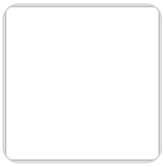
ChatGPT said:

You said:



ChatGPT said:

You said:



ChatGPT said:

You said:

like for before.mp4 what do i record exactly

ChatGPT said:

You said:

should i show even q?

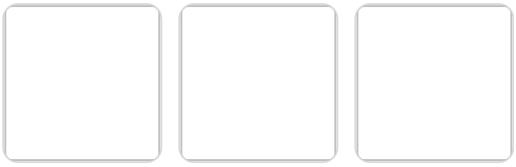
ChatGPT said:

You said:

before.mp4
File

ChatGPT said:

You said:



after running main.py(before)

ChatGPT said:

You said:

```
import random

SIZE = 4


class Board:
    def __init__(self):
        self.grid = [[0] * SIZE for _ in range(SIZE)]
        self.score = 0
        self.add_random_tile()
        self.add_random_tile()

    def add_random_tile(self):
        empty = [(r, c) for r in range(SIZE) for c in range(SIZE) if
self.grid[r][c] == 0]
        if empty:
            r, c = random.choice(empty)
            self.grid[r][c] = 4 if random.random() < 0.1 else 2

    @staticmethod
    def slide_line(line):
        values = [x for x in line if x]
        result = []
        for value in values:
            if result and result[-1] == value:
                result[-1] *= 2  # intentional double-merge bug
            else:
                result.append(value)
        return result + [0] * (SIZE - len(result))

    def move_left(self):
        changed = False
        for r in range(SIZE):
```

```
            old = self.grid[r][:]
            self.grid[r] = self.slide_line(old)
            changed |= old != self.grid[r]
        return changed

    def move_right(self):
        changed = False
        for r in range(SIZE):
            old = self.grid[r][:]
            self.grid[r] = list(reversed(self.slide_line(list(reversed(old)))))
            changed |= old != self.grid[r]
        return changed

    def move_up(self):
        changed = False
        for c in range(SIZE):
            old = [self.grid[r][c] for r in range(SIZE)]
            new = self.slide_line(old)
            for r in range(SIZE):
                self.grid[r][c] = new[r]
            changed |= old != new
        return changed

    def move_down(self):
        changed = False
        for c in range(SIZE):
            old = [self.grid[r][c] for r in range(SIZE)]
            new = list(reversed(self.slide_line(list(reversed(old)))))
            for r in range(SIZE):
                self.grid[r][c] = new[r]
            changed |= old != new
        return changed

    def can_move(self):
        if any(0 in row for row in self.grid):
            return True
        for r in range(SIZE):
            for c in range(SIZE):
                if c + 1 < SIZE and self.grid[r][c] == self.grid[r][c + 1]:
                    return True
                if r + 1 < SIZE and self.grid[r][c] == self.grid[r + 1][c]:
                    return True
        return False
```

ChatGPT said:

You said:

next

ChatGPT said:

You said:

```
from board import Board


class Game:
    def __init__(self):
        self.board = Board()
        self.best_score = 0
        self.history = []

    def display(self):
        print("\n" + "+------+------+------+------+")
        for row in self.board.grid:
            print("|" + "|".join(f"{x:^6}" if x else f"{' ':^6}" for x in row) +
"|")
            print("+------+------+------+------+")
        print("Score:", self.board.score, " Best:", self.best_score)

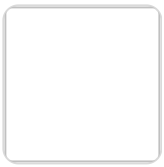
    def move(self, key):
        moves = {"a": self.board.move_left, "d": self.board.move_right,
                 "w": self.board.move_up, "s": self.board.move_down}
        if key not in moves:
            return False
        changed = moves[key]()
        if changed:
            self.board.add_random_tile()
        return changed

    def run(self):
        print("2048 — W/A/S/D to move, U to undo, Q to quit.")
        while True:
            self.display()
            if any(2048 in row for row in self.board.grid):
                print("You reached 2048!")
                return
```

```
        if not self.board.can_move():
            print("No legal moves remain.")
            return
        key = input("> ").strip().lower()
        if key == "q":
            return
        if key == "u":
            print("Undo is not implemented yet.")
            continue
        if key not in "wasd":
            print("Use W/A/S/D.")
            continue
        if self.move(key):
            self.best_score = max(self.best_score, self.board.score)
```

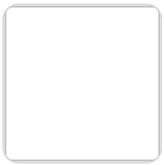
ChatGPT said:

You said:



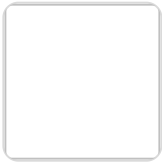
ChatGPT said:

You said:



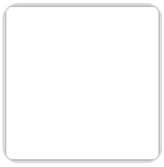
ChatGPT said:

You said:



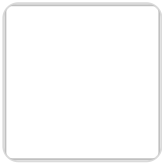
ChatGPT said:

You said:



ChatGPT said:

You said:



ChatGPT said:

You said:

```
import random

SIZE = 4


class Board:
    def __init__(self):
        self.grid = [[0] * SIZE for _ in range(SIZE)]
        self.score = 0
        self.add_random_tile()
        self.add_random_tile()

    def add_random_tile(self):
        empty = [(r, c) for r in range(SIZE) for c in range(SIZE) if
self.grid[r][c] == 0]
        if empty:
            r, c = random.choice(empty)
            self.grid[r][c] = 4 if random.random() < 0.1 else 2


    @staticmethod
    def slide_line(line):
        values = [x for x in line if x]
        result = []
        merged = []

        for value in values:
```

```
            if result and result[-1] == value and not merged[-1]:
                result[-1] *= 2
                merged[-1] = True
            else:
                result.append(value)
                merged.append(False)

        return result + [0] * (SIZE - len(result))


    def move_left(self):
        changed = False
        for r in range(SIZE):
            old = self.grid[r][:]
            self.grid[r] = self.slide_line(old)
            changed |= old != self.grid[r]
        return changed

    def move_right(self):
        changed = False
        for r in range(SIZE):
            old = self.grid[r][:]
            self.grid[r] = list(reversed(self.slide_line(list(reversed(old)))))
            changed |= old != self.grid[r]
        return changed

    def move_up(self):
        changed = False
        for c in range(SIZE):
            old = [self.grid[r][c] for r in range(SIZE)]
            new = self.slide_line(old)
            for r in range(SIZE):
                self.grid[r][c] = new[r]
            changed |= old != new
        return changed

    def move_down(self):
        changed = False
        for c in range(SIZE):
            old = [self.grid[r][c] for r in range(SIZE)]
            new = list(reversed(self.slide_line(list(reversed(old)))))
            for r in range(SIZE):
                self.grid[r][c] = new[r]
            changed |= old != new
        return changed
```

```
def can_move(self):
    if any(0 in row for row in self.grid):
        return True
    for r in range(SIZE):
        for c in range(SIZE):
            if c + 1 < SIZE and self.grid[r][c] == self.grid[r][c + 1]:
                return True
            if r + 1 < SIZE and self.grid[r][c] == self.grid[r + 1][c]:
                return True
    return False
```

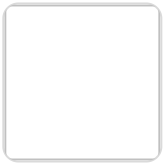
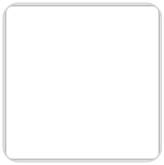
ChatGPT said:

You said:

do one thing give me the moves i should make so that testing becomes easier

ChatGPT said:

You said:

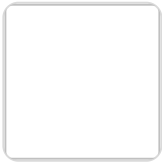
ChatGPT said:

You said:

next

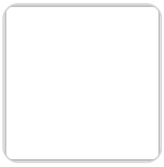
ChatGPT said:

You said:



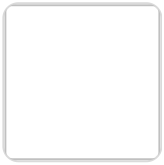
ChatGPT said:

You said:



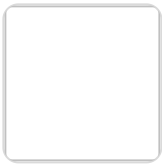
ChatGPT said:

You said:



ChatGPT said:

You said:



ChatGPT said:

You said:

done now lets complete all the pending deliverables for lab4

ChatGPT said:

You said:

done i added all next

ChatGPT said:

You said:

i created lab4 folder locally in file explorer and its not even on vs code currently

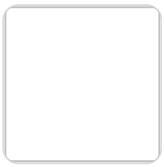
ChatGPT said:

You said:

I created Lab-4 directly on my Desktop.

ChatGPT said:

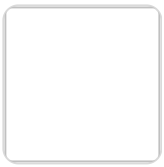
You said:



actually this is the case 4 of us just push our content into respective labs

ChatGPT said:

You said:



what else should it have

ChatGPT said:

You said:

we didnt update readme content right?

ChatGPT said:

You said:

Scenario 14 — 2048

A terminal 2048 implementation with directional movement and tile merging.

Provided files

- `main.py` — entry point.
- `game.py` — game loop, commands, and game state.
- `board.py` — grid movement and tile logic.
- `requirements.txt` — dependency declaration.

Setup

```bash
python main.py
```

No package installation is required.

Before changing the code

Run the starter and deliberately construct repeated-tile rows if possible. Read the
board movement code and trace one complete move from input to tile creation.

Task 1 — Correct merge semantics

Fix the merge algorithm so an original tile can participate in at most one merge
during a single move. Preserve normal compression and movement behaviour.

Done when: repeated patterns such as four equal tiles produce the standard 2048
result, not a chain merge of a newly created tile in the same move.

Task 2 — Finish game-state handling

Implement win detection, no-move detection, and correct tile creation semantics.
A move that leaves the board unchanged must not create a new tile.

Done when: reaching 2048 ends the game, a full board with no legal moves ends the
game, and unchanged moves do not alter the board.

Task 3 — Undo and best score

Add a one-level undo. Restoring a move must restore both board contents and score.
Track the best score for the current run.

Done when: one undo reverses exactly one successful move and does not create a tile.

Task 4 — Move feedback

Add concise feedback for successful moves and merges. Do not trigger feedback merely
because internal board-scanning functions were called.

Done when: one accepted move produces one action-level result and invalid/unchanged
moves do not claim that a move occurred.

Required testing

Test ordinary slides, four-equal patterns, separated equal pairs, unchanged moves,
2048 detection, no-move boards, undo, score restoration, invalid commands, and quitting.

LLM usage

You may use an LLM during the lab. The goal is to use it as a coding assistant while
retaining responsibility for understanding and testing the result.

- Inspect the existing code before asking for changes.
- Ask for explanations when you do not understand a proposed change.
- Test generated code against the stated behaviour and edge cases.
- Keep your complete LLM chat history for submission.
- Do not replace the whole project with an unrelated

implementation.
- Keep all state in memory; do not add CSV, JSON, SQLite, or other persistence.

Submission checklist

- [ ] Task 1 completed and the original defect was reproduced and fixed.
- [ ] Tasks 2–4 completed and tested.
- [ ] Boundary and invalid-input cases tested.
- [ ] No unnecessary external dependencies added.
- [ ] No persistent storage added.
- [ ] Code remains understandable and modular.
- [ ] Complete LLM chat-history link included.

Folder structure

```text
scenario-02-2048/
├── README.md
├── requirements.txt
├── main.py
├── game.py
└── board.py
```

Submission Checklist

Submission is only the following three things:

- [ ] A 10-second video of gameplay ****before**** your changes, showing the bug/broken behavior
- [ ] A 10-second video of gameplay ****after**** your changes, showing the bug fixed and the new features working
- [ ] The Chat/LLM used page link, with the complete chat history

ChatGPT said:

You said:

where do i add the chat link in readme and i came across a cross checklist in readme which does not look professional,so update the readme with a provision for me to paste the link(make sure the evaluator can click onto the link i attach on readme). then guide me regarding the testing part

ChatGPT said:

You said:

got the right ouputs post testing what do i do next?

ChatGPT said:

You said:

test 2 rightly done

ChatGPT said:

You said:

what do i do next

ChatGPT said:

You said:

finished what next?

ChatGPT said:

You said:

it says score:0, best:0; no legal moves remain

ChatGPT said:

You said:

do i need to show all these testing in after.mp4?

ChatGPT said:

You said:

after.mp4
File

check this after.mp4 out and let me know if any changes to be made,if not lets go ahead with final deliverables(creating the common lab4 folder in the main repo our group shares,then add my content inside that folder in an organised manner,so that the repo looks clean and professional)

Worked for 28s
Analysis paused

ChatGPT said:

‹/›

You said:

The lab4 git should be like lab4(common for all 4 of us) followed by individual folders with all the deliverables inside that particular folder

ChatGPT said: